Distributed Computing Paradigms

Distributed computing is one of the most vibrant areas in computer science. There is an ongoing evolution of new technologies for supporting network applications, bringing with them new conceptual models and terminologies

3.1 Paradigms and Abstraction

Abstraction

The most fundamental concept in computer science, *abstraction* is the idea of encapsulation, or detail hiding. *We often use abstraction when it* is *not necessary to know the exact details of how something works or* is *represented, because we can still make use of it in its simplified form. Getting involved with the detail often tends to obscure what we are trying to understand, rather than illuminate* it. *Abstraction plays a very important role in programming because we often want to model, in software, simplified versions of things that exist* in *the real world, without having to build the real things.*

In software engineering, abstraction is realized with the provision of tools or facilities that allow software to be built without the developer having to be cognizant of some of the underlying complexities. It is not an overstatement to say that the tools for abstraction are the force behind modern-day software, and they exist in every aspect of application development. As examples, we make use of compilers to abstract the detail of the machine languages, and Java programmers use the *Abstract Window Toolkit (AWT)* to rapidly develop graphic displays. In the area of distributed applications, there has been an explosion of tools and facilities based on a wide variety of paradigms that offer varying degrees of abstraction.

Paradigms

Webster's Dictionary defines the word *paradigm* as "a pattern; example, or model." In the study of any subject of great complexity, it is useful to identify the basic patterns or models and classify the details according to these models.

The paradigms will be presented in the order of their level of abstraction. At the lowest level of abstraction is message passing, which encapsulates the least amount of detail. Object space occupies the other extreme of the spectrum, as it is the most abstract of all the paradigms.

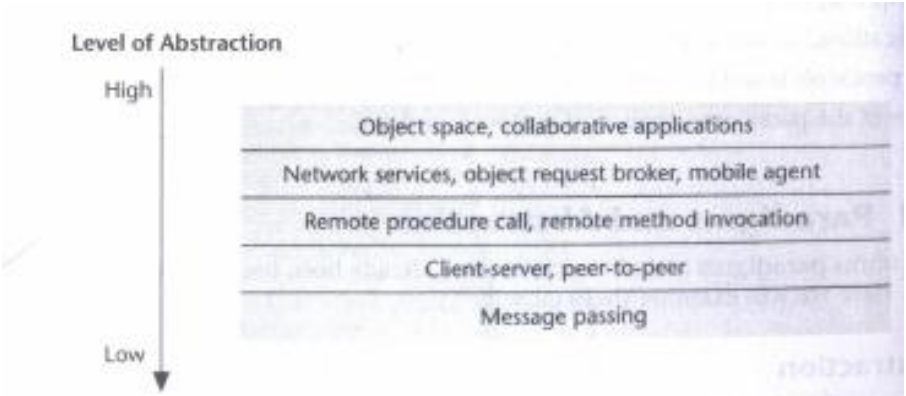


Figure 3.1 Distributed computing paradigms and their level of abstraction.

3.2 **An Example Application**

Throughout the discussion that follows, a common application will be used to illustrate how each paradigm may be applied. ***The example application is an online auctioning system***. We will simplify the system to one that handles only one auctioned item per session. During each auctioning session, an item is open for bids placed by the auction participants. At the end of a session, the auctioneer announces the outcome. In the description of the various implementations, we will confine our attention to the distributed computing aspects of the service layer of the application's architecture.

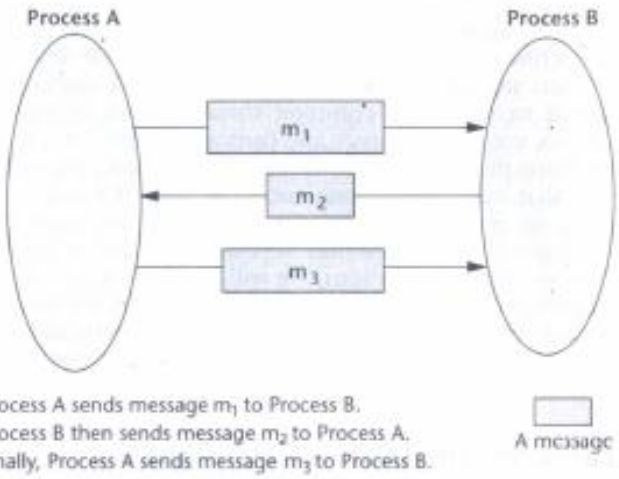
3.3 **Paradigms for Distributed Applications**

Message Passing

The basic approach to interprocess communications is message passing. In this paradigm, data representing messages are exchanged between two processes, a sender and a receiver. Messagepassing is the most fundamental paradigm for distributed applications.

A process sends a message representing a request. The message is delivered to a receiver, which processes the request and sends a message in response. In turn, the reply may trigger a further request, which leads to a subsequent reply and so forth. The base operations required to support the message-passing paradigm are *send* and *receive.* For connection-oriented communication the operations *connect* and *disconnect* are also required.

The socket application program interface is based on this paradigm. Using a socket, a logical construct, two processes may exchange data as follows: A sender writes or inserts a message into the socket: at the other end, a receiver reads or extracts a message from the socket.



gure 3.2 The message-passing paradigm.

The Client-Server Paradigm

Perhaps the best known paradigm for network applications, the client-server model assigns asymmetric roles to two collaborating processes. One process, the *server,* plays the role of a service provider, waiting passively for the arrival of requests. The other, the *client,* issues specific requests to the server and awaits the server's response. Simple in concept, the client-server model provides an efficient abstraction for the delivery of network services. Operations required include those for a server process to listen and to accept requests, and for a client process to issue requests and accept responses. By assigning asymmetric roles to the two sides, event synchronization is simplified: The server process waits for requests, and the client in turn waits for responses.

Many Internet services support client-server applications. These services are often known by the protocol that the application implements. Well-known Internet services include HTIP, FTP, DNS, finger, gopher etc.

The two basic models that we have looked at so far, *client-server* and *message passing,* are sufficient as a basis for the implementation of our auctioning system. Each participant, as well as the auctioneer program, assumes the role both *client* and *server,* as follows:

For session control:

• As a server, a participant waits to hear an announcement from the auctioneer (1) when the session starts, (2) whenever there is an update on the current highest bid, and (3) when the session ends.

• As a client, the auctioneer sends a request that announces the three types of events outlined m the previous item.

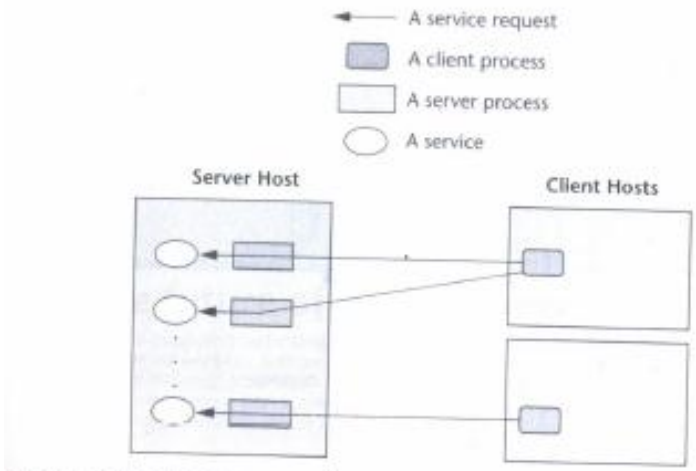


Figure 3.3 The client-server paradigm.

For accepting bids:

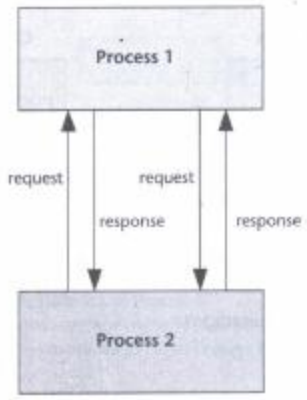
• As a client, a participant sends a new bid to a server.

• As a server, an auctioneer accepts new bids and updates the current highest bid.

The Peer-to-Peer Paradigm

In the client-server paradigm the participating processes play different roles: Client processes issue requests while the server processes listen passively for service requests and provide the requested service in response. In the peer-to-peer paradigm the participating processes play equal rules, with equivalent capabilities and responsibilities.

Each participant may issue a request to another participant and receive a response. A well-known example of a peer-to-peer file transfer service is *Napster.com;* similar sites allow files (primarily audio files) to be transmitted among computers on the Internet.



Whereas the client-server paradigm is an ideal model for a centralized network service (where a server process provides the service and client processes access the service through the server), the peer-to-peer paradigm is more appropriate for applications such as instant messaging, peer-to-peer file transfers, video conferencing, and collaborative work. It is also

possible for an application to be based on both the client-server model and the peer-to-peer model.

Examples of such protocols and their accompanying tools include the JXTA project and Jabber an XML-based, open protocol for instant messaging and presence. The implementation of our auctioning system can be simplified significant with the availability of a tool that supports the peer-to-peer paradigm. A participant can now contact the auctioneer directly to register for the auction. The auctioneer subsequently contacts each participant to initiate the auction session, during which individual participants may obtain the latest status and submit a bid. At the conclusion of the auction, the winning bidder can be notified by the auctioneer, and other participants may learn about the outcome by contacting the auctioneer.

The Message System Paradigm

The Message System or Message-Oriented Middleware (MOM) paradigm is an elaboration of the basic message-passing paradigm.

In this paradigm, a message system serves as an intermediary among separate, independent processes. The message system acts as a switch for messages, through which processes exchange messages asynchronously, in a decoupled manner.

A sender deposits a message with the message system, which forwards it to a message queue associated with each receiver. Once a message is sent, the sender is free to move on to other tasks.

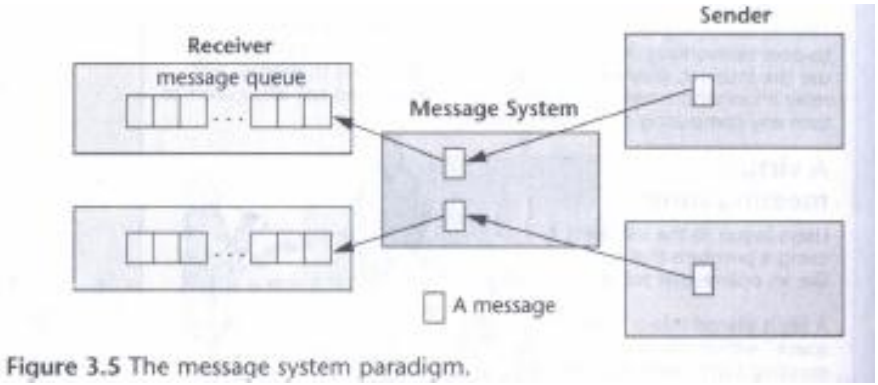


Figure 3.5 The message system paradigm.

The Point-to-Point Message Model

In this model, a message system forwards a message from the sender to the receiver's message queue. Unlike the basic message passing model, the middleware provides a message depository and allows the sending and the receiving be decoupled. Via the middleware, a sender deposits a message in the message queue of the receiving process. A receiving process extracts the messages from its message queue and handles each message accordingly.

Compared to the basic message-passing model, the point-to-point message passing paradigm provides an additional abstraction for asynchronous operations to achieve the same effect with basic message passing, a developer would have make use of threads or child processes.

This implementation of our auctioning system using the point-to-point message model is the same as with the basic message-passing model. The only difference is that messages are channeled through the middleware and the sending and receiving are thereby decoupled.

The Publish/Subscribe Message Model

In this model, each message is associated with a specific topic or event. Applications interested in the occurrence of a specific event may subscribe to messages for that event. When the awaited event occurs, the process publishes a message announcing the event or topic. The middleware message system distributes the message to all its subscribers.

The publish/subscribe message model offers a powerful abstraction for multicasting or group communication. The *publish* operation allows a process to multicast to a group of processes, and the *subscribe* operation allows a process to listen for such a multicast.

Using the publish/subscribe message model, the implementation of our auctioning system may proceed as follows:

• Each participant subscribes to a *begin-auction* event message.

• The auctioneer signifies the beginning of the auctioning session by sending a *begin-auction* event message.

• Upon receiving the *begin-auction* event, a participant subscribes to an *end-auction* event message.

• The auctioneer subscribes to messages for *new-bid* events.

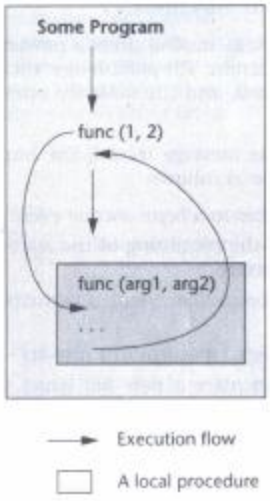
• A participant wishing to place a new bid issues a *new-bid* event message, which will be forwarded to the auctioneer.

• At the end of the session, the auctioneer issues an *end-auction* event message to inform all participants of the outcome. If desired, additional message events can be added to allow the participants to monitor the status of the auction.

Remote Procedure Call Model

The message-passing model works well for basic network protocols and for basic network applications. But, as applications grew increasingly complex, it became necessary for further abstractions to be provided for network programming. In particular, it was desirable to have a paradigm that allows distributed software to be programmed in a single processor in a manner similar to conventional applications that run on a single processor.

Remote Procedure Call (RPC) model provides such an abstraction. Using this model, communication between two processes is carried out using a concept similar to that for a local procedure call, which is familiar to application.



A remote procedure call involves two independent processes, which may reside on separate machines. A process, *A,* wishing to make a request to another process, *B,* issues a procedure call to *B,* passing with the call a list of argument values. As in the case of local procedure calls, a Remote Procedure Call triggers a predefined action in a procedure provided by process *B.* At the completion of the procedure, process *B* returns a value to process *A.*

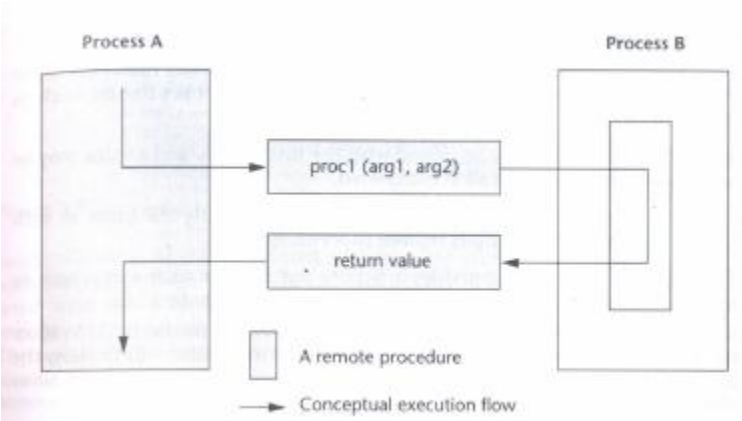


Figure 3.7 The Remote Procedure Call paradigm.

A procedure call is made by one process to another, with data passed as arguments. Upon receiving a call, the actions encoded in the procedure are executed as a result.

As a comparison the message-passing model is *data-oriented,* with the actions triggered by the, message exchanged, while the RPC mode I'IS *action-oriented,* and with the data passed as arguments.

RPC allows programmers to build network applications using a programming construct similar to the local procedure call, providing a convenient abstract for both interprocess communications and event synchronization.

Since its introduction in the early 1980s, the Remote Procedure Call model has been widely used in network applications. There are two prevalent APIs for Remote Procedure Calls: One, the Open Network Computing Remote Procedure Call, evolved from the RPC API originated from Sun Microsystems in the early 1980s. The other well-known API is the Open Group Distributed Computing Environment (DCE) RPC. In addition, there is the Simple Object Access Protocol (SOAP), and its implementations support Webbased Remote Procedure Calls.

Using RPC to implement our auctioning system will proceed as follows:

• The auctioning program provides a remote procedure for each participant to register itself and another procedure for a participant to make a bid.

• Each participant program provides the following remote procedures: (1) to allow the auctioneer to call a participant to announce the onset of the session (2) to allow the auctioneer to inform a participant of a new highest bid and (3) to allow the auctioneer to announce the end of the session.

The Distributed Objects Paradigms

The idea of applying object orientation to distributed applications is a natural extension of object-oriented software development. Applications access objects distributed over a network. Objects provide methods, through the invocation of an application obtains access to services.

Remote Method Invocation

Remote Method Invocation (RMI) (Figure 3.8) is the object-oriented equivalent of the Remote Procedure Call. In this model, a process invokes the methods in an object, which may reside on a remote host. As with RPC, arguments may be passed with the invocation, and a value may be returned when the method call is completed.

The implementation of our auctioning system is essentially the same as with RPC, except that object methods replace procedures:

• The auctioning program provides a remote method for each participant to register itself and another method for a participant to make a bid.

• Each participant program provides the following remote methods: (1) to allow the auctioneer to call it to announce the onset of the session, (2) to allow the auctioneer to inform

the program of a new highest bid, and (3) to allow the auctioneer to announce the end of the session.

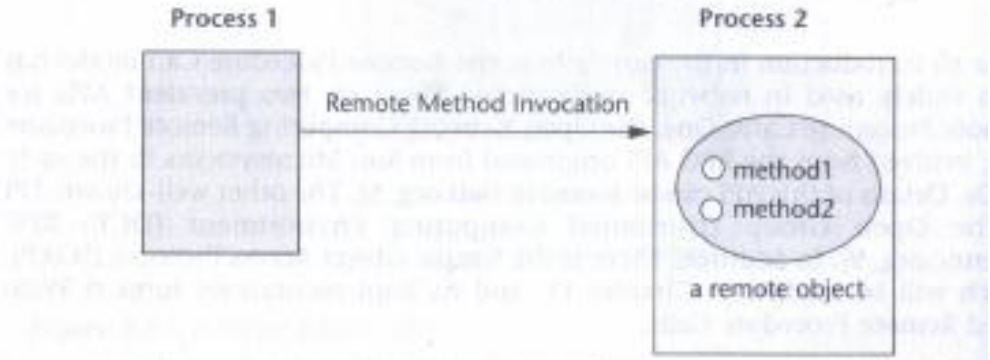


Figure 3.8 The Remote Method Invocation paradigm.

The Object Request Broker Paradigm

In the object broker paradigm a process issues requests to an object request broker (ORB), which directs the request to an appropriate object that provides the desired service. The paradigm closely resembles the Remote Method Invocation model in its support for remote object access, The difference is that the object request broker in this paradigm functions as middleware, allowing an application, as an object requestor, to potentially access multiple remote (or local) objects. The request broker may also function as a mediator for heterogeneous objects, allowing interactions among objects implemented using different APIs and *lor* running on different platforms.

Implementation of the auctioning system using ORBs is similar to using RMI, with the exception that each object (auctioneer or participant) must be registered with the ORB and requested from the ORB. Each participant issues requests to the auctioneer object to register for the session and to make bids.

Through the ORB, the auctioneer invokes the methods of each participant to announce the start of the session, to update the bidding status, and to announce the end of the session.

This paradigm is the basis of the Object Management Group's CORBA (Common Object Request Broker Architecture).Toolkits based on the architecture include Java Interface Definition Language (Java IDL), Microsoft COM, Microsoft DCOM, Java Bean, and Enterprise Java Bean, are also based on distributed-object paradigms, since the components are essentially specialized, packaged objects designed to interact with each other through standardized interfaces. In addition, application servers, popular for enterprise applications, are middleware facilities that provide access to objects or components.

The Object Space

Perhaps the most abstract of the object-oriented paradigms, the object space paradigm assumes the existence of logical entities known as object spaces. The participants of an application converge in a common object space. A provider places objects as entries into an object space, and requestors who subscribe to the space may access the entire the paradigm.

In addition to the abstractions provided by other paradigms, the object space paradigm provides a virtual space or meeting room among providers and requestors of network resources, as objects.

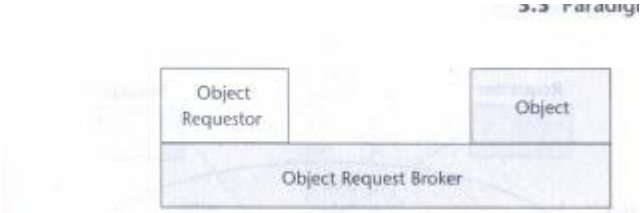


Figure 3.9 The object request broker paradigm.

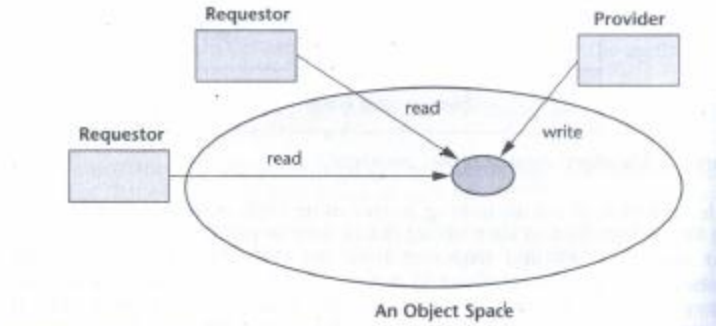


Figure 3.10 The object space paradigm.

For the auctioning system, all participants as well as the service provider subscribe to a common object space. Each participant deposits an object into the object space to register for the session and to be notified when the auctioning session starts. At the onset of the session, the auctioneer deposits an object into the object space. The object contains the item information and the bid history.

A participant wishing to place a bid retrieves the object from the space and, if he/she so chooses, places a new bid in the object before returning the object to the space. At the end of the session, the auctioneer retrieves the object from the space and contacts the highest bidder.

An existing toolkit based on this paradigm is javaSpaces. The Mobile Agent Paradigm A mobile agent is a transportable program or object. In this paradigm, an agent is launched from an originating host. The agent then travels autonomously from host to host according to an itinerary that it carries. At each stop, the agent accesses the necessary resources or services

and performs the necessary tasks to accomplish its mission. The paradigm offers the abstraction for a transportable program or object. In lieu of message exchanges, data is carried by the program/object as the program is itself transported among the participants.

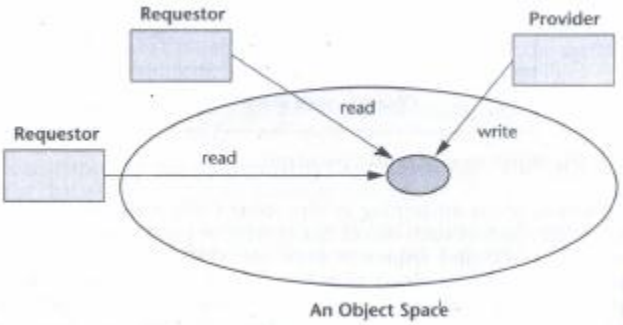
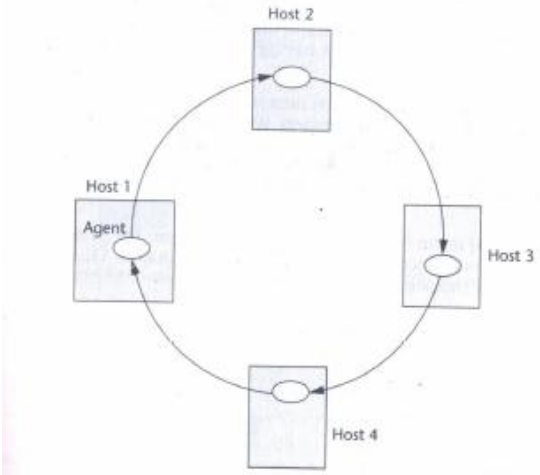


Figure 3.10 The object space paradigm.

The mobile agent paradigm provides a novel way of implementing our auctioning system. At the onset, each participant launches a mobile agent to the auctioneer. The mobile agent carries with it the identity, including the network address, of the participant that it represents. Once the session starts, the auctioneer launches a mobile agent that carries with it an initial details of the participants, as well as the current highest bid. The mobile agent circulates am on



gure 3.11 The mobile agent paradigm.

Other participants and the auctioneer until the session ends, at which time the auctioneer launches the agent to make one more round among the participants to announce the outcome.

Location transparency allows a software developer to access an object or a service without having to be cognizant of the location of the object or service.

The implementation of our auctioning system is the same as under the RMI paradigm except that the auctioneer registers itself with the directory service, allowing the participants to

locate it and, once the session has commenced, to make bids. The participants provide callback methods to allow the auctioneer to announce the start and the end of the session, and to update the status of the session.

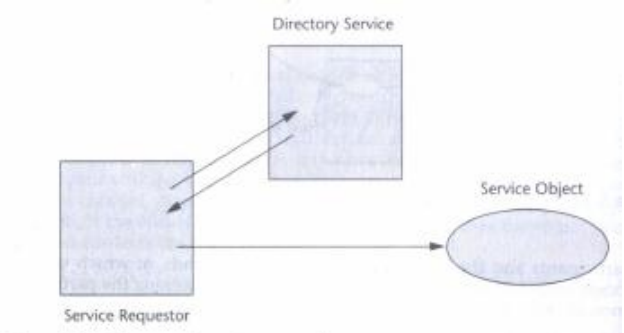


Figure 3.12 The network services paradigm.

The Collaborative Application (Groupware) Paradigm

In this model for computer-supported cooperative work, processes participate in a collaborative session as a group. Each participating process may contribute input to part or all of the group. Processes may do so using multicasting to send data to all or part of the group, or they may use virtual sketchpads or whiteboards, which allow each participant to read and write data to a shared display.

To implement the auctioning system using the message-based groupware paradigm, the auctioneer initiates a group, to be joined by all interested participants. At the onset of the session, the auctioneer multicasts a message announcing the start. During the session, each bid is multicast to all participants so that each participant access the status of the auction.

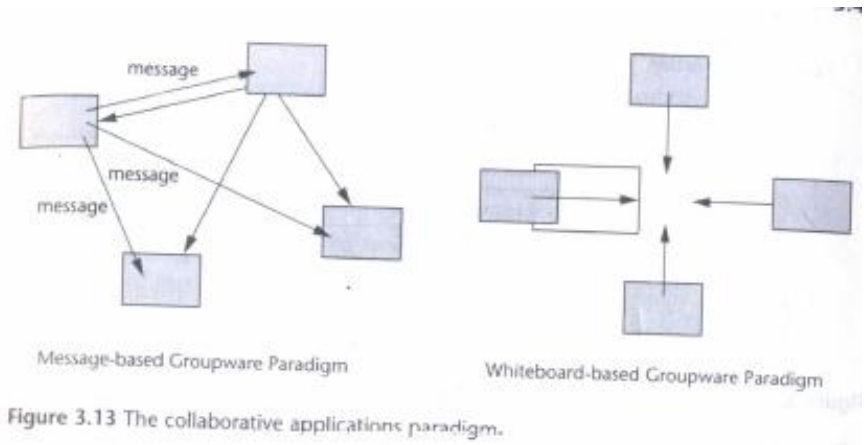


Figure 3.13 The collaborative applications paradigm.

Finally, the auctioneer terminates the session by multicasting a message announcing the outcome.

Trade offs

In order to choose the most appropriate form of paradigm for the suitable for the situation, one must be aware of the trade-offs between the different approaches. Suffice it to say that each paradigm or tool has some advantage over another Each advantage, however, may be offset by some disadvantage. In the following paragraphs we will look at some of the issues that should be considered.

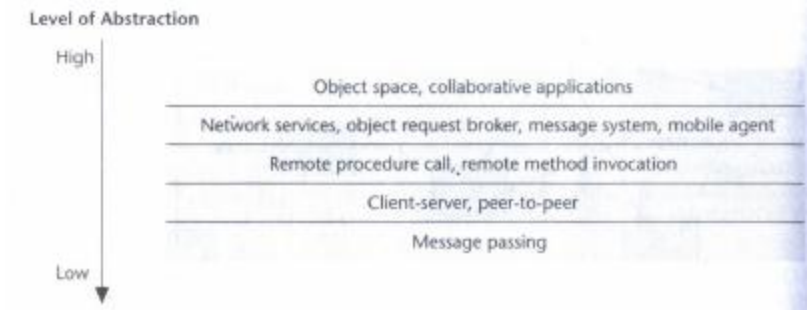
Level of Abstraction versus Overhead

At the lowest levels are the most basic paradigms: *message passing* and *client server.* At the highest level are *object spaces* and *collaborative computing* which provide the most abstraction. The development of a highly complex application may be greatly aided by a tool that offers a high level of abstraction.

But abstraction comes at a price: *overhead.*

Considering the *Remote Method Invocation* paradigm additional objects called known as *stubs* and *skeletons* need to be present at run time to handle the run time details of the interprocess communications. Due to this the software modules require additional system resources and execution time.

For this reason, the socket API may be the most appropriate application that calls for a fast response time or for a system abstraction.



ure 3.14 The paradigms and levels of abstraction.

Scalability

The complexity of a distributed application increases significantly, possibly exponentially, as the number of participants (processes or objects) increases.

Consider our example application, the auctioning system. As described, the auctioneer must manage the addresses of the participants so that it may announce the start and end of the session. Furthermore, the auctioneer needs to repeatedly contact individual participants to inform them of the latest highest bid.

Using message passing, the developer must provide the coding to manage the addresses and to contact the participants individually. The complexity grows as the number of participants increases.

With a toolkit based on a high-level paradigm such as *object spaces* or a *publish/ subscribe message system,* the complexity of managing the participants is handled by the system. An application thus implemented can accommodate an increase in the number of participants with no additional complexity.

Mobile-agent is another paradigm that allows an application to scale well, since the number of participating hosts can increase without any significant impact on the complexity of a program based on the paradigm.

Cross-Platform Support

Paradigms, being abstract models, are inherently platform independent. Toolkits based on paradigms, on the other hand, may be platform dependent, and many are. In order to provide the generality, a tool that supports heterogeneous platforms necessarily incurs complexity, compared to one that supports a single platform. For the same reason, the programming syntax also tends to be more cumbersome.

Many of the Java technologies, including Java RMI API and JavaSpaces, by choice run on Java virtual machines only.

Beyond these trade-Offs, there are software-engineering issues that should be considered when we are choosing a tool. Some of these are

- The maturity and stability of the tool
- The fault tolerance provided by the tool
- The availability of developer tools
- Maintainability
- Code reuse